

浙江大学

本科生毕业设计

开题报告



学生姓名 彭超琦

学生学号 3220105744

指导教师 朱霖潮

年级与专业 2022 级 计算机科学与技术

所在学院 计算机科学与技术学院

一、题目：面向科学场景的智能体工具调用优化研究

二、指导教师对开题报告的具体要求：

文献综述需批判性梳理已有研究，指出不足并说明本文切入点，引用优先选取近五年高质量成果。方法须与研究问题匹配，数据来源清晰，分析逻辑严密。

指导教师（签名）_____

年 月 日

目 录

一、开题报告 1

1. 问题提出的背景 错误! 未定义书签。

1.1 背景介绍 1

1.2 本研究的意义和目的 2

2. 论文的主要内容和技術路线 3

2.1 主要研究内容 3

2.2 技术路线 6

2.3 可行性分析 8

3. 研究计划进度安排及预期目标 9

3.1 进度安排 9

3.2 预期目标 9

4. 参考文献 10

二、外文翻译 12

三、外文原文 25

一、开题报告

1. 问题提出的背景

1.1 背景介绍

1.1.1 大语言模型与智能体技术的发展现状

近年来，大语言模型（Large Language Model, LLM）技术实现了跨越式发展[1][2]，以 GPT 系列、通义千问、文心一言等为代表的模型，在自然语言理解、逻辑推理、任务规划等方面的能力得到了显著提升，为人工智能从感知智能向决策智能的跨越奠定了基础[1]。

在此背景下，智能体（Agent）技术成为大模型落地应用的核心方向——通过赋予大模型工具调用、自主规划、多轮迭代的能力，智能体能够突破大模型的知识边界与能力局限，对接外部系统与工具，完成复杂的现实世界任务，成为当前人工智能领域的研究与产业落地热点[3][4][5]。

目前，国内外已形成了成熟的智能体开发基础设施，包括 LangChain、LlamaIndex 等开源框架[7][8]，提供了标准化的工具封装、任务规划、记忆管理等核心能力，大幅降低了智能体系统的开发门槛。同时，主流大模型均已原生支持函数调用（Function Calling）能力[2]，能够精准解析用户需求，生成符合格式要求的工具调用指令，为智能体工具调用能力的落地提供了底层支撑[4][5]。

1.1.2 AI for Science 领域的发展与工具需求

AI for Science（人工智能赋能科学研究）已成为人工智能与基础科学交叉融合的核心赛道，被广泛认为是加速科学发现、提升科研效率的核心技术手段[10][11]。当前，科学研究场景中存在大量重复性、流程化的工作，包括数值计算、统计分析、实验数据可视化、文献检索、专业数据处理等，这些工作普遍依赖各类专业软件、开源计算库与分析工具，存在工具种类繁多、接口格式不统一、使用门槛高、操作流程繁琐等问题。

对于科研人员而言，尤其是非计算机专业的科研工作者，往往需要花费大量时间学习各类工具的使用方法、处理不同工具之间的数据兼容问题，无法将核心精力聚焦于科学问题本身。因此，科研领域对能够自动化调用各类工具、完成标准化科研流程的智能系统，存在迫切的现实需求。具备工具调用能力的科学智能体，能够有效降低专业工具的使用门槛，实现科研流程的自动化执行，成为 AI for Science 领域的重要研究方向[6][10]。

1.1.3 当前存在的主要问题

当前主流的智能体框架与工具调用技术，在通用场景中已得到了广泛应用[7][8]，但在科学研究场景中仍存在明显的适配性缺陷，无法满足科研工作的专业性、严谨性要求，核心问题集中于三个方面：第一，缺乏面向科学领域的统一工具抽象规范。当前科学计算领域的工具种类繁多，不同开源库、专业软件的接口格式、参数要求、数据返回格式差异极大，通用智能体框架缺乏标准化的工具封装规范，导致工具集成成本高、适配难度大，无法快速扩展支持各类科研工具。第二，科学任务理解与拆解能力不足。科学研究任务具有专业性强、逻辑严谨、流程固定的特点，通用智能体的任务规划机制无法精准理解专业科研需求[3][9]，难以将模糊的科研问题拆解为严谨、可执行的工具调用序列，易出现任务拆解逻辑错误、工具匹配不当等问题。第三，工具调用的容错性与结果可靠性不足。科研工作对计算结果的精度与严谨性有极高要求，而当前大模型工具调用普遍存在参数幻觉、调用逻辑错误、异常处理能力弱等问题[4][5]，缺乏针对科学场景的结果校验机制，无法保障输出结果的准确性与可靠性，难以满足科研工作的合规要求。

1.2 本研究的意义和目的

1.2.1 研究意义

本研究针对科学场景的核心需求，构建适配科研任务特性的智能体工具调用优化体系，完善垂直领域智能体工具调用的理论框架，弥补现有通用智能体框架在科学场景下的适配性缺陷，为科研领域智能体系统的设计与开发提供可参考的实现范式，丰富 AI for Science

领域的技术体系[10][11]。

本研究设计的系统能够有效降低科研人员使用专业计算工具的门槛，实现数值计算、统计分析、数据可视化等高频科研任务的自动化执行，大幅减少科研工作中的重复性劳动，提升科研工作效率。同时，系统采用模块化、可扩展的设计思路，能够快速适配生物信息学、材料科学、环境科学等多个细分科研领域的需求[11]，具备较强的落地应用潜力，能够为基层科研工作者提供轻量化、易用型的科研辅助工具。

1.2.2 研究目的

本研究旨在设计并实现一套面向科学场景的智能体工具调用优化系统，解决当前智能体在科研场景中工具适配难、任务拆解不准、调用容错性差、结果不可靠等核心问题[3][4][5]。通过标准化的工具封装、专项优化的任务拆解机制、轻量化的调用优化与容错校验方案，提升智能体在科学场景中工具调用的准确率、成功率与执行效率，最终实现科研任务的全流程自动化执行，为科研人员提供稳定、易用的智能辅助工具。

2. 论文的主要内容和技術路线

2.1 主要研究内容

本研究聚焦科学场景智能体工具调用的核心痛点，设置 4 项核心研究内容，具体如下：

2.1.1 面向科学场景的工具抽象与统一接口规范设计

本部分旨在搭建标准化科学工具封装体系，针对性解决当前科研领域工具接口不统一、跨工具集成难度大、调用与执行规范杂乱等问题，通过系统化架构设计与模块化落地，实现各类科研工具的规范化整合、通用化调用与安全化运行，为科研系统的工具联动、高效研发筑牢底层技术支撑，整套体系形成规范统一、模块封装、安全管控闭环衔接的完整架构。为从根源统一工具调用标准，首先要设计通用科学工具抽象基类，明确规范工具名称、功能描述、参数格式、执行逻辑、结果返回形式及异常处理规则等核心要素，打破不同领域、不同类型科研工具的接口壁垒，让各类工具调用一致接口，减少接口差异带来的集成障碍，

降低二次开发与跨工具适配成本[4][7]。

基于 Python 科学计算生态，聚焦科研场景封装核心工具模块，搭建可扩展的科学工具库，覆盖四大核心类别：数值计算类基于 NumPy、SciPy[12]，封装线性代数运算、积分微分、函数拟合等基础计算工具；统计分析类基于 Pandas、SciPy[12][13]，整合数据清洗、描述性统计、假设检验、回归分析等全流程统计工具；数据可视化类基于 Matplotlib、Seaborn，标准化封装折线图、散点图、热力图等科研常用绘图工具；基础专业科研类基于 Biopython，封装生物信息学基础序列分析工具。四大模块既可独立使用，也能协同完成复杂科研任务。搭建工具白名单机制与参数前置校验模块，拦截大模型生成的非法工具调用请求，校验输入参数合规性，过滤不符合规范的参数内容，从源头规避工具执行异常，保障系统运行安全性与稳定性，支持体系常态化高效落地使用。

2.1.2 科学任务理解与拆解机制优化

本部分旨在解决当前智能体处理专业科研任务时，普遍存在的任务拆解逻辑混乱、关键步骤遗漏、配套工具匹配不当、执行路径无法落地等痛点，全面提升智能体对科研场景的适配能力与任务处理精准度，为智能科研系统的高效调度执行筑牢核心支撑[3][9]。

搭建多轮迭代的层级化任务拆解框架，针对复杂科研任务实现“主任务→细分原子任务→可执行单步操作”的三级精细化拆解，确保每一个最小执行单元都能直接对接前文构建的标准化科学工具库，支持单个或组合工具调用，规避无对应工具支撑、无法落地执行的有效拆解步骤，强化任务拆解的可操作性[3][6]。

结合科研领域的专业特性深化 Prompt 工程优化[9]，融入多类型典型科研任务实操案例、细分领域基础科研规则与通用行业规范，进一步提升大模型对专业科研需求的语义理解与场景识别准确率，降低任务拆解错误率与工具误匹配率，使智能体能够稳定适配基础科研与专业细分领域的各类任务处理需求[2][9]。

2.1.3 工具调用优化与容错校验机制设计

本部分聚焦科研场景下工具调用全流程优化，搭建容错与校验体系，核心目标是提升工具调用成功率、执行效率与结果可靠性，针解决当前科学工具调用容错性差、跨工具协同不畅、无效调用冗余、输出结果缺乏严谨性核验的突出问题，为整套智能科研系统的稳定落地与高效运行筑牢关键支撑[4][5][6]。

基于启发式规则构建动态工具选择与组合策略，基于任务拆解后的细化执行需求，综合考量工具功能适配度、执行效率与调用成本，匹配最优工具组合及标准化调用顺序，主动剔除重复冗余的调用步骤，压缩无效执行环节，减少资源损耗，提升整体任务执行效率[3][5]。

搭建跨工具统一数据流管理模块，制定中间数据格式规范，打通不同工具间的数据传输壁垒，实现前序工具输出结果与后序工具输入参数的自动格式转换、无缝适配，解决多工具协同执行中的数据不兼容、格式不匹配等问题，保障链式工具调用的连贯性与流畅性[6][7]。

搭建覆盖全调用流程的容错处理与结果双重校验机制，一方面构建自动化异常处理闭环，针对调用失败、参数报错、执行中断等问题，完成错误原因自动溯源、参数智能修正、按需重试执行，避免单次异常导致整体任务中断[4][5]；另一方面嵌入专业科研校验规则，封装统计显著性检验、数据合理性筛查、结果逻辑验证等专项工具，对输出结果开展自动化复核，严格把控结果质量，确保所有输出内容完全符合科研工作的严谨性与规范性要求[10][11]。

2.1.4 系统原型实现与场景验证

本部分是研究的落地验证环节，旨在验证系统的可用性与优化效果。

完成各模块的集成开发，构建完整的系统原型，实现用户科研需求输入→任务自动拆解→工具调用执行→结果校验输出的全流程自动化运行，并完成系统的全流程联调与 bug 修复。

构建科学任务测试数据集，涵盖 20 个不同类型、不同难度的典型科研任务，设计对比实验，以任务成功率、工具调用准确率、平均执行步数为核心指标，对比本研究优化后的系统与原生 GPT-4 直接调用[2]、通用 LangChain Agent [7]的性能差异，验证优化方案的有效性。

选取真实科研场景，包括生物序列基础分析、实验数据统计与可视化、材料性能数据拟合与分析等，完成系统的场景化验证，测试系统在真实科研场景中的可用性与实用性。

2.2 技术路线

本研究基于 Python 技术栈与主流开源框架完成系统开发。整体技术路线遵循循序渐进、闭环迭代的原则，划分为 6 大核心实施环节，具体实施流程如下。

2.2.1 前期筹备与文献调研梳理

本环节完成领域文献系统性调研与技术选型落地。

梳理大语言模型工具调用、智能体框架构建及 AI for Science 领域国内外核心文献、顶会顶刊研究成果与优质开源项目[1][3][4][10]，总结现有研究的优势亮点与现存短板，完成开题报告的撰写、修订与审核。完成全套技术选型，初步确定适配科研场景的开发环境与技术栈体系：底层大模型基于 OpenAI GPT API 实现核心推理能力[2]，备选国产通义千问 API；智能体基础框架采用 LangChain[7]，实现工具调用与任务调度逻辑；科学计算模块基于 xNumPy、SciPy、Pandas、Matplotlib、Biopython 等模块[12][13]。

2.2.2 标准化科学工具库封装实现

本环节完成核心科学工具库的落地开发与测试。

完成数值计算、统计分析、数据可视化、基础专业科研四大类高频科研工具的封装，并分别针对每一项独立工具编写各自对应的单元测试代码，验证工具功能的正确性与稳定性。

完成工具白名单管控模块与参数前置校验模块开发,实现非法调用与异常参数的前置拦截[5],完成工具库整体联调测试,确保整套工具库满足后续系统调用的稳定性需求。

2.2.3 科学任务拆解机制优化开发

本环节开展科研任务拆解模块的优化与开发工作。

设计适配科学任务的思维链 Prompt 模板与层级化任务拆解模板[9],搭建标准化任务拆解流程,完成模块核心代码开发。构建专项测试任务集,对 Prompt 模板进行多轮调试与迭代优化,重点提升智能体对科研任务的理解准确率、拆解逻辑严谨性,减少无效拆解与逻辑漏洞,保障任务拆解结果可落地、可执行[2][3]。

2.2.4 工具调用与容错校验模块研发

本环节完成动态工具选择与组合策略、跨工具数据流管理模块、自动化异常重试机制、科研结果合规性校验模块的开发工作。

针对每个独立模块开展专项功能测试与异常场景测试,验证动态工具匹配效率、跨工具数据适配能力、异常处理响应速度与结果校验精准度[4][5][6],确保各模块功能达标、运行稳定,为系统全流程流畅执行提供保障。

2.2.5 系统集成与实验验证分析

本环节整合集成前期开发的所有独立模块,搭建完整的系统原型,开展全流程联调测试,排查并修复运行过程中出现的漏洞与 bug,优化系统运行流畅度与交互可用性[7][8]。

构建涵盖多类科研场景的标准化测试数据集,设计对照实验与性能验证实验,采集并整理实验数据,量化分析系统优化效果。选取真实科研场景开展落地验证[10][11],根据测试反馈进一步优化系统适配性与实用性,确保系统满足实际科研工作需求。

2.2.6 论文撰写与最终成果定稿

完成毕业论文（设计）全文撰写，涵盖研究背景与意义、国内外相关工作综述、系统整体设计、核心模块实现、实验设计与结果分析、研究结论与展望等核心章节。结合导师指导意见，对论文内容进行多轮修订完善，同步补充优化系统源码、实验数据、测试报告等配套毕设材料，最终完成论文定稿与全部毕业设计材料的整理归档。

2.3 可行性分析

2.3.1 技术可行性

本研究所有核心功能基于成熟的开源框架与技术栈实现。底层大模型采用成熟商用 API，具备稳定的函数调用能力[2]；智能体核心逻辑基于 LangChain 框架实现[7]，该框架已被广泛应用于各类智能体系统开发，文档完善、社区活跃，确认可以实现核心功能；科学工具基于 Python 科学计算开源库封装[12][13]，这些库经过长期的工业界与学术界验证，功能稳定、文档齐全。整体技术方案成熟可靠，技术可行性高。

2.3.2 资源可行性

本研究底层大模型采用商用 API，通过按量付费的方式使用；相关的文献资料、开发教程、开源案例等可通过知网、IEEE Xplore、GitHub、官方文档等公开渠道获取[7][8]。资源可行性高。

2.3.3 工作量与实现难度可行性

本研究核心开发工作包括工具封装、Prompt 工程、规则模块开发、系统集成等，预计可在 12 周内完成核心系统的开发与测试；实验环节需构建测试数据集、完成对比实验，无需大量的算力消耗。同时，本人具备丰富等 Python 开发、科学计算库使用、大模型 API 调用等相关经验，能够胜任系统开发工作。工作量与实现难度符合要求。

3. 研究计划进度安排及预期目标

3.1 进度安排

本研究整体周期为 16 周。具体进度安排如下所示。

阶段	时间周期	工作内容	预期结果
1	第 1-2 周	完成文献调研与选题论证，确定技术路线与研究边界，完成开题报告的撰写、修改与提交	开题报告终稿、文献调研清单
2	第 3-4 周	完成科学工具抽象基类设计，完成四大类核心科研工具的封装与单元测试，完成工具库整体联调	科学工具库代码、单元测试报告
3	第 5-6 周	完成科学任务拆解 Prompt 模板设计，实现任务拆解模块的开发，完成模块调试与准确率优化	Prompt 模板集、任务拆解模块代码、模块测试报告
4	第 7-8 周	完成动态工具选择、数据流管理、异常重试、结果校验模块的开发与测试，完成各核心优化模块的联调	核心优化模块代码、模块功能测试报告
5	第 9-10 周	完成全系统的集成与联调，构建科学任务测试数据集，进行对比实验，整理实验数据与分析结果	系统原型、测试数据集、对比实验报告
6	第 11-12 周	完成科研场景下的验证测试，优化系统可用性，按照学校论文规范，撰写毕业论文初稿	优化后的系统原型、毕业论文初稿
7	第 13-14 周	根据导师意见修改毕业论文，同步优化系统与实验内容，并继续修改撰写毕业论文	论文修改稿、完善后的系统与实验材料、查重报告
8	第 15-16 周	完成毕业论文终稿定稿，整理所有毕设材料，制作答辩 PPT，准备答辩	论文终稿、完整毕设材料、答辩 PPT

表 3.1 研究进度安排表

3.2 预期目标

完成一套面向科学场景的标准化工具抽象框架，封装不少于 12 个高频科研工具，覆盖数值计算、统计分析、数据可视化、基础专业科研分析四大类别，形成可扩展的科学工具库，并确保所有工具均通过单元测试，可稳定执行。

设计完成适配科学任务的 Prompt 模板集[9], 实现复杂科研任务的层级化拆解, 任务拆解逻辑准确率不低于 85%, 能够匹配对应的工具调用步骤。

完成工具调用优化与容错校验机制的开发, 对比原生 GPT-4 直接调用模式[2], 系统的工具调用整体成功率提升至 90% 以上, 任务平均执行步数减少 20% 以上, 具备完善的异常处理与结果校验能力。

完成面向科学场景的智能体工具调用优化系统原型开发, 实现科研需求输入到结果输出的全流程自动化运行, 在 3-5 个真实基础科研场景中完成可用性验证。

形成完整的毕设材料, 包括系统源代码、使用说明文档、开发文档、测试数据集、实验报告等, 具备可复现性。

完成毕业论文, 内容完整、逻辑严谨、数据详实。

4. 参考文献

- [1] Brown, T. et al. Language Models are Few-Shot Learners. NeurIPS 2020
- [2] OpenAI. GPT-4 Technical Report. 2023
- [3] Yao, S. et al. ReAct: Synergizing Reasoning and Acting in Language Models. 2022
- [4] Schick, T. et al. Toolformer: Language Models Can Teach Themselves to Use Tools. 2023
- [5] Qin, Y. et al. Tool Learning with Foundation Models. 2023
- [6] Shen, Y. et al. HuggingGPT: Solving AI Tasks with ChatGPT and its Friends. 2023
- [7] Chase, H. LangChain Documentation.
- [8] Liu, J. et al. LlamaIndex: Data Framework for LLM Applications.
- [9] Wang, X. et al. Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning. 2023
- [10] Jumper, J. et al. Highly accurate protein structure prediction with AlphaFold. Nature, 2021
- [11] Butler, K. T. et al. Machine learning for molecular and materials science. Nature, 2018

[12] Harris, C. R. et al. Array programming with NumPy. Nature, 2020

[13] McKinney, W. Data Structures for Statistical Computing in Python (Pandas). 2010

二、外文翻译

反应式推理与行动融合：大语言模型中的推理与行动协同机制

1 引言

人类智能的一个独特特征，是能够将面向任务的行动与语言推理无缝结合。有理论认为，这种能力在人类认知中扮演着关键角色，能够实现自我调节与策略制定，并维持工作记忆。以厨房做菜为例，在两个具体的操作步骤之间，我们会通过语言进行推理：追踪操作进度（“食材都切好了，该烧水煮锅了”）、处理突发状况或根据实际情况调整计划（“没有盐了，那就用酱油和胡椒粉替代吧”），并判断何时需要获取外部信息（“怎么揉面团？我去网上查一下”）。我们也会通过行动（翻开食谱看做法、打开冰箱检查食材）为推理提供支撑，解答当下的问题（“现在能用现有食材做什么菜？”）。这种“行动”与“推理”的深度协同，让人类即便在从未遇到的场景中，或是面对信息不确定的情况时，也能快速学习新任务，做出稳健的决策与推理。

近期的研究成果表明，在自主系统中融合语言推理与交互式决策具备可行性。一方面，经过合理提示的大语言模型（LLMs）展现出了一种涌现能力——能够生成多步推理轨迹，从而在算术、常识推理和符号推理任务中推导出答案。但这种“思维链”推理是一个静态的黑箱：模型仅依靠自身的内部表征生成推理过程，并未与外部世界建立关联，这使其无法进行动态的反应式推理，也无法更新自身知识，进而容易出现事实幻觉和推理过程中的错误传播问题（图 1（1b））。另一方面，已有研究探索将预训练语言模型应用于交互式环境中的规划与行动决策，这类研究的核心是通过语言先验预测行动。这些方法通常将多模态观测信息转换为文本，利用语言模型生成特定领域的行动或计划，再通过控制器选择并执行。但它们并未让语言模型对高层目标进行抽象推理，也未借助工作记忆为行动提

供支撑；仅有黄等人（2022b）的研究实现了一种简易的语言推理，用于重述与当前状态相关的空间事实。除了这类与简单积木交互的具身任务外，目前尚无研究探索如何将推理与行动以协同的方式融合，应用于通用任务求解；也未验证这种融合相比单独的推理或行动，是否能带来系统性的性能提升。

本研究提出了反应式推理与行动融合范式（ReAct），这是一种将大语言模型的推理与行动能力相结合的通用范式，可用于解决各类语言推理与决策任务（图 1）。ReAct 通过提示大语言模型，以交替的方式生成与任务相关的语言推理轨迹和行动指令，让模型能够进行动态推理，从而制定、维护并调整面向行动的高层计划（为行动而推理）；同时，模型还能与外部环境（如维基百科）交互，将外部信息融入推理过程（为推理而行动）。

我们在四类不同的基准数据集上，对 ReAct 和当前最优的基线方法进行了实证评估：问答任务、事实验证任务、文本基游戏任务和网页导航任务。在 HotPotQA 和 FEVER 任务中，模型可调用维基百科 API 进行交互，实验结果显示，ReAct 的性能优于纯行动生成模型，且与思维链推理（CoT）方法的性能相当。而整体性能最优的方法，是将 ReAct 与 CoT 相结合的融合策略——该策略能让模型在推理过程中同时利用内部知识和外部获取的信息。在 ALFWorld 和 WebShop 任务中，仅需单次或两次 ReAct 提示，模型的性能就能超越经过 $10^3 \sim 10^5$ 个任务实例训练的模仿学习或强化学习方法，成功率分别实现了 34% 和 10% 的绝对提升。此外，通过对比纯行动的受控基线模型，我们验证了稀疏、灵活的推理在决策中的重要性，ReAct 在这一方面展现出了持续的优势。除了通用性强、性能提升显著外，推理与行动的融合还让模型在所有任务领域中，都具备更好的可解释性、可信度和可诊断性：人类能够清晰区分模型的结论是来自内部知识还是外部环境，也能通过检查推理轨迹，理解模型做出行动决策的依据。

本研究的核心贡献如下：（1）提出 ReAct 这一全新的基于提示的范式，实现大语言模型中推理与行动的协同，适用于通用任务求解；（2）在多种基准数据集上开展大量实验，

验证了在少样本学习设置下, ReAct 相比单独进行推理或行动生成的传统方法具有显著优势; (3) 通过系统性的消融实验与分析, 阐明了行动在推理任务中的作用, 以及推理在交互式任务中的价值; (4) 分析了基于提示设置的 ReAct 存在的局限性 (即对推理和行动行为的支撑能力有限), 并通过初步的微调实验, 验证了 ReAct 在增加训练数据后具备进一步性能提升的潜力。将 ReAct 扩展至更多任务的训练与部署, 并与强化学习等互补范式相结合, 有望进一步释放大语言模型的潜能。

2 反应式推理与行动融合: 推理与行动的协同机制

考虑智能体与环境交互以完成任务的通用场景: 在时间步 t , 智能体从环境中接收观测信息 $o_t \in \mathcal{O}$, 并根据策略 $\pi(a_t | c_t)$ 执行行动 $a_t \in \mathcal{A}$, 其中 $c_t = (o_1, a_1, \dots, o_{t-1}, a_{t-1}, o_t)$ 是智能体的上下文信息。当上下文到行动的映射 $c_t \mapsto a_t$ 高度隐含且需要大量计算时, 学习该策略将极具挑战性。例如, 图 1 (1c) 中的智能体无法生成正确的最终行动 (行动 4) 来完成问答任务, 原因是该行动需要对轨迹上下文 (问题、行动 1-3、观测 1-3) 进行复杂推理。同理, 图 1 (2a) 中的智能体无法从上下文中判断出 1 号洗手池内没有 1 号胡椒瓶, 因此持续做出存在事实幻觉的行动。

ReAct 的核心思想十分简洁: 我们将智能体的行动空间扩展为 $\mathcal{A}^L = \mathcal{A} \cup \mathcal{L}$, 其中 $\mathcal{L}$ 为语言空间。语言空间中的行动 $a^L_t \in \mathcal{L}$ (我们将其称为思维或推理轨迹) 不会对外部环境产生影响, 因此也不会得到观测反馈。相反, 思维 a^L_t 的作用是通过当前上下文 c_t 进行推理, 整合有用信息, 并更新上下文 $c_{t+1} = (c_t, a^L_t)$, 为后续的推理或行动提供支撑。如图 1 所示, 有效的思维形式具有多样性, 例如: 拆解任务目标并制定行动计划 (2b, 行动 1; 1d, 思维 1)、融入与任务求解相关的常识知识 (2b, 行动 1)、从观测信息中提取关键内容 (1d, 思维 2、4)、追踪任务进度并调整行动计划 (2b, 行动 8)、处理突发状况并修正行动计划 (1d, 思维 3) 等。

但由于语言空间 L 具有无限性，在这种扩展的行动空间中进行学习难度较大，且需要强大的语言先验知识。本研究主要聚焦于冻结大语言模型的设置：以 PaLM-540B 为基础模型，通过少样本的上下文示例进行提示，让模型同时生成特定领域的行动和自由形式的语言思维，以完成任务（图 1 (1d)、(2b)）。每个上下文示例都是人类为解决某一任务实例所生成的行动、思维与环境观测的轨迹（详见附录 C）。对于以推理为核心的任务（图 1 (1)），我们让模型交替生成思维和行动，使任务求解轨迹由多个“思维 - 行动 - 观测”步骤构成。而对于可能需要执行大量行动的决策任务（图 1 (2)），思维仅需在轨迹中最关键的位置稀疏出现即可，因此我们让语言模型自主决定思维与行动的异步生成时机。

由于决策与推理能力被整合进同一个大语言模型，ReAct 具备以下四大独特特征：A）设计直观、易于实现：ReAct 的提示词设计十分简便，标注人员仅需在自身的行动步骤旁，用语言记录下推理思路即可。本研究未使用任何特殊的格式设计、思维模板或示例筛选策略，各任务的提示词设计细节将在第 3、4 节详细说明。B）通用性强、灵活适配：得益于灵活的思维空间和思维 - 行动生成格式，ReAct 可适用于各类具有不同行动空间和推理需求的任务，包括但不限于问答、事实验证、文本游戏和网页导航。C）性能优异、鲁棒性高：仅通过 1~6 个上下文示例进行学习，ReAct 就能对新的任务实例展现出良好的泛化能力，在不同领域中，性能持续优于仅进行推理或仅进行行动的基线模型。我们还将第 3 节验证 ReAct 在微调后的额外性能增益，并在第 4 节说明 ReAct 的性能对提示词选择具有较强的鲁棒性。D）人机对齐、可调控性：ReAct 实现了可解释的序列决策与推理过程，人类能够轻松检查推理过程的合理性和事实的正确性。此外，人类还能通过编辑思维内容，实时控制或修正智能体的行为（详见第 4 节图 5）。

3 知识密集型推理任务

我们首先将 ReAct 应用于多跳问答、事实验证等知识密集型推理任务。如图 1(1d) 所示，通过与维基百科 API 交互，ReAct 能够检索信息为推理提供支撑，同时通过推理确定下一步的检索目标，实现了推理与行动的协同。

3.1 实验设置

任务领域

我们选取两个对知识检索和推理能力要求较高的数据集展开实验：（1）HotPotQA，一个多跳问答基准数据集，要求模型对至少两段维基百科文本进行推理以得出答案；（2）FEVER，一个事实验证基准数据集，根据是否存在可验证该声明的维基百科文本，将每个声明标注为“支持”“反驳”或“信息不足”。本研究中，两个任务均采用仅问题输入的设置：模型仅接收问题 / 声明作为输入，无法获取支撑文本，必须依靠内部知识，或通过与环境交互检索知识来完成推理。

行动空间

我们设计了一个简易的维基百科网页 API，包含三类行动，以支持交互式信息检索：（1）search[实体]：若该实体的维基百科页面存在，返回页面的前 5 个句子；若不存在，由维基百科搜索引擎推荐相似度最高的 5 个实体。（2）lookup[字符串]：返回页面中包含该字符串的下一个句子，模拟浏览器的“查找 (Ctrl+F)”功能。（3）finish[答案]：提交答案并完成当前任务。需要说明的是，该行动空间仅能根据精确的页面名称检索文本的少量内容，其检索能力远弱于当前最优的词汇检索器或神经检索器。我们设计该简易 API 的目的，是模拟人类使用维基百科的方式，迫使模型通过显式的语言推理完成检索。

3.2 实验方法

ReAct 提示方法

在 HotPotQA 和 FEVER 任务中，我们分别从训练集中随机选取 6 个和 3 个任务实例，人工构建 ReAct 格式的轨迹，作为提示词中的少样本示例。与图 1 (d) 一致，每个轨迹

由多个“思维 - 行动 - 观测”步骤构成（即密集型思维），自由形式的思维被用于多种推理场景，具体包括：拆解问题（“我需要先检索 X，找到 Y，再查找 Z”）、从维基百科观测信息中提取关键内容（“X 创立于 1844 年”“该段落未提及 X”）、进行常识推理（“X 并非 Y，因此 Z 一定是……”）或算术推理（“ $1844 < 1989$ ”）、指导检索策略调整（“或许我可以换个关键词，检索 / 查找 X”），以及整合信息得出最终答案（“…… 因此答案是 X”）。更多细节详见附录 C。

基线模型

我们通过对 ReAct 轨迹进行系统性的消融，构建了多种基线模型的提示词（格式见图 1 (1a-1c)）：（a）标准提示（Standard）：移除 ReAct 轨迹中的所有思维、行动和观测信息。（b）思维链提示（CoT）：移除 ReAct 轨迹中的行动和观测信息，作为仅推理的基线模型。我们还构建了自洽性思维链基线（CoT-SC）：推理阶段将解码温度设为 0.7，采样 21 条思维链轨迹，选取出现次数最多的答案作为最终结果，该方法被证实能持续提升思维链模型的性能。（c）仅行动提示（Act）：移除 ReAct 轨迹中的思维信息，该方法与 WebGPT 通过互联网交互解答问题的方式相似，但二者的任务场景、行动空间不同，且 WebGPT 采用模仿学习和强化学习，而非提示学习。

内部与外部知识的融合策略

我们将在 3.3 节详细说明：ReAct 的问题求解过程更贴合事实、更具实际依据，而 CoT 在构建推理结构方面更准确，但容易产生事实幻觉或不合理的推理。因此，我们提出将 ReAct 与 CoT-SC 相结合的策略，让模型根据以下启发式规则，自主决定切换至另一种方法：A) ReAct→CoT-SC：若 ReAct 在设定步数内未得出答案，则切换至 CoT-SC。我们为 HotPotQA 和 FEVER 分别设定 7 步和 5 步的阈值，实验发现增加步数无法提升 ReAct 的性能。B) CoT-SC→ReAct：若 n 条 CoT-SC 采样轨迹中，出现次数最多的答案的占比不足 $n/2$ （即模型的内部知识无法为任务提供足够可靠的支撑），则切换至 ReAct。

微调方法

由于大规模人工标注推理轨迹和行动的成本极高，我们采用与泽利克曼等人（2022）相似的自举法：利用 ReAct（及其他基线模型）生成的 3000 条含正确答案的轨迹，对较小的语言模型（PaLM-8B/62B）进行微调，让模型能够根据输入的问题 / 声明，解码生成完整的轨迹（包含所有思维、行动和观测信息）。更多细节详见附录 B.1。

3.3 实验结果与分析

ReAct 性能持续优于仅行动模型

表 1 展示了以 PaLM-540B 为基础模型，不同提示方法在 HotPotQA 和 FEVER 任务上的实验结果。可以发现，ReAct 在两个任务上的性能均优于仅行动模型，证明了推理对行动的指导价值——尤其是在整合信息得出最终答案的阶段（见图 1 (1c-d)）。微调实验的结果也进一步证实，推理轨迹能让模型的行动决策更具依据。

ReAct 与 CoT 的性能对比

另一方面，ReAct 在 FEVER 任务上的性能优于 CoT（60.9 vs. 56.3），在 HotPotQA 任务上的性能略低于 CoT（27.4 vs. 29.4）。FEVER 任务中，“支持” / “反驳” 类声明的差异往往十分细微（详见附录 D.1），因此通过行动检索准确、最新的知识至关重要。为了更深入地分析 ReAct 和 CoT 在 HotPotQA 任务上的行为差异，我们从两个模型的结果中，各随机抽取 50 条含正确答案和错误答案的轨迹（以精确匹配率 EM 为评判标准），共 200 个示例，并人工标注了它们的成功与失败模式（表 2）。核心发现如下：

- A) 事实幻觉是 CoT 的核心问题：在成功求解的案例中，CoT 的假阳性率远高于 ReAct（14% vs. 6%）；而事实幻觉也是 CoT 最主要的失败原因，占比达 56%。相比之下，由于能访问外部知识库，ReAct 的问题求解过程更贴合实际、以事实为驱动，可信度更高。
- B) 思维 - 行动 - 观测的交替结构存在局限性：这种结构虽提升了 ReAct 的事实依据性和可信度，但也降低了其构建推理步骤的灵活性，导致 ReAct 的推理错误率高于 CoT。

我们发现 ReAct 存在一种典型的错误模式：模型会重复生成之前的思维和行动，无法推理出合理的下一步行动，也无法跳出循环，我们将这类错误归为“推理错误”。C) 有效检索是 ReAct 成功的关键：无信息价值的检索是 ReAct 的主要失败原因之一，占比 23%——这类检索会干扰模型的推理过程，使其难以恢复推理思路并调整检索策略。这一现象可视为模型事实性与灵活性的权衡，也正是我们提出 ReAct 与 CoT 融合策略的动因。各成功与失败模式的示例详见附录 E.1。此外，我们还发现 HotPotQA 的部分问题存在答案标签过时的情况（示例见图 4）。

ReAct 与 CoT-SC 融合是提示学习的最优策略

表 1 的结果还显示，HotPotQA 和 FEVER 任务上性能最优的提示方法分别为 ReAct→CoT-SC 和 CoT-SC→ReAct。此外，图 2 展示了不同方法的性能随 CoT-SC 采样数的变化趋势：尽管两种融合策略各在一个任务上表现最优，但在所有采样数下，它们的性能均显著且持续优于纯 CoT-SC；仅需 3~5 个采样数，融合策略就能达到纯 CoT-SC 21 个采样数的性能。这些结果表明，将模型的内部知识与外部知识合理融合，对推理任务具有重要价值。

ReAct 是微调学习的最优方法

图 3 展示了四种方法（标准提示、CoT、仅行动、ReAct）在 HotPotQA 任务上的提示学习 / 微调学习的缩放效应。以 PaLM-8B/62B 为基础模型时，由于从上下文示例中同时学习推理和行动的难度较大，ReAct 在提示学习中的性能排名最后。但仅通过 3000 个示例进行微调后，ReAct 成为四种方法中性能最优的模型：微调后的 PaLM-8B-ReAct 性能优于所有 PaLM-62B 的提示学习模型，微调后的 PaLM-62B-ReAct 性能甚至优于所有 PaLM-540B 的提示学习模型。与之相反，标准提示和 CoT 的微调效果远差于 ReAct 和仅行动模型：前两者本质上是让模型记忆（可能存在幻觉的）知识事实，而后两者则是教模型如何（通过推理）行动，从维基百科中获取信息——这是一种更具泛化性的知识推

理能力。由于所有提示学习方法的性能，仍与领域内的最优方法存在显著差距（表 1），我们认为，利用更多人工标注数据进行微调，是释放 ReAct 潜能的更优方式。

4 决策任务

我们还将 ReAct 应用于两类基于语言的交互式决策任务 ——ALFWorld 和 WebShop。这两个任务的环境均具有复杂性，要求智能体在稀疏奖励的条件下，完成长序列的行动决策，因此亟需通过推理为行动和探索提供有效指导。

ALFWorld 任务

ALFWorld（图 1 (2)）是一个合成的文本基游戏环境，与具身智能体基准数据集 ALFRED 相适配。该环境包含 6 类任务，要求智能体通过文本行动与模拟家居环境交互、导航（如“走到 1 号咖啡桌旁”“拿起 2 号纸”“打开 1 号台灯”），完成高层目标（如“在台灯下查看纸张”）。每个任务实例包含超过 50 个场景位置，专家策略需要执行 50 余步才能完成，因此对智能体的子目标规划、追踪能力，以及系统性探索能力（如“逐个检查所有桌子，寻找台灯”）提出了很高要求。ALFWorld 的一个核心挑战，是要求智能体判断家居物品的可能位置（如“台灯通常在桌子、架子或梳妆台上”），这一特点让大语言模型能够充分利用预训练的常识知识。

在 ReAct 提示词设计中，我们为每类任务从训练集中随机标注 3 条轨迹，每条轨迹包含稀疏的思维，主要用于：（1）拆解总目标；（2）追踪子目标的完成情况；（3）确定下一个子目标；（4）通过常识推理判断物品的可能位置及操作方式。ALFWorld 的提示词示例详见附录 C.4。遵循施里达尔等人（2020b）的实验设置，我们在 134 个未见过的评估游戏上，采用按任务类型分组的方式进行评估。为验证模型的鲁棒性，我们从 3 条标注轨迹中选取 2 条进行排列组合，为每类任务构建 6 个不同的提示词。仅行动模型的提示词基于相同的轨迹构建，仅移除思维信息 —— 由于任务实例从训练集中随机选取，该设

置对 ReAct 和仅行动模型均无偏向性，为验证稀疏思维的价值提供了公平、可控的对比。

基线模型选用 BUTLER，这是一个模仿学习模型，每类任务均经 10^5 条专家轨迹的训练。

WebShop 任务

ReAct 能否在充满噪声的真实世界语言环境中，完成实际应用任务？我们基于 WebShop 展开研究 —— 这是一个近期提出的在线购物网页环境，包含 118 万件真实商品和 1.2 万条人类指令。与 ALFWorld 不同，WebShop 包含大量结构化和非结构化文本（如从亚马逊爬取的商品标题、描述、规格选项），要求智能体通过网页交互（如“检索‘带抽屉的床头柜’”、点击“颜色：现代镍白”或“返回检索页”等按钮），根据用户指令选购商品（如“我想找一个带抽屉的床头柜，表面为镍质工艺，价格低于 140 美元”）。该任务的评估指标为平均得分（所有任务中，选中商品满足用户需求属性的比例均值）和成功率（选中商品满足用户所有需求的任务占比），实验在 500 条测试指令上开展。

仅行动模型的提示词包含检索、选择商品、选择规格、购买四类行动；ReAct 的提示词则在此基础上，增加了推理步骤，用于判断探索方向、购买时机，以及与指令相关的商品规格。提示词示例详见表 6，模型预测结果详见附录表 10。基线模型选用模仿学习（IL）方法。

实验结果

ReAct 在 ALFWorld（表 3）和 WebShop（表 4）两个任务上的性能均优于仅行动模型。

在 ALFWorld 任务中，ReAct 的最优实验结果实现了 71% 的平均成功率，显著优于仅行动模型的最优结果（45%）和 BUTLER 模型（37%）；即便 ReAct 的最差实验结果（48%），也优于后两者的最优结果。此外，在 6 组受控实验中，ReAct 的性能均持续优于仅行动模型，相对性能提升幅度在 33%~90% 之间，平均提升 62%。从定性分析来看，仅行动模型由于缺乏思维推理，无法将总目标正确拆解为子目标，也无法追踪环境的当前状态。

ReAct 与仅行动模型的轨迹对比示例详见附录 D.2.1 和 D.2.2。

在 WebShop 任务中，仅需单次提示的仅行动模型，性能已与模仿学习、模仿学习 + 强化学习方法相当；而增加了稀疏推理的 ReAct，性能实现了显著提升，成功率相比此前的最优结果实现了 10% 的绝对提升。通过分析示例发现，ReAct 能够通过推理，弥补噪声观测信息与行动之间的差距，更准确地识别与指令相关的商品和规格（如“对于‘客厅用省空间的软包脚凳’这一需求，该商品有 39×18×18 英寸的尺寸和蓝色款可选，符合购买要求”）。但现有方法的性能仍与人类专家存在显著差距（表 4）——人类专家会进行更多的商品探索和检索词调整，而这正是基于提示学习的模型目前难以实现的。

内部推理与外部反馈的价值

据我们所知，ReAct 是首个将大语言模型的推理与行动融合，应用于闭环系统交互式环境的研究。与之最接近的前期研究是《内心独白》，该研究让具身智能体的行动由同名的“内心独白”驱动。但 IM 的“内心独白”仅局限于对环境状态的观测，以及对智能体完成目标所需步骤的描述；而 ReAct 在决策任务中的推理轨迹更灵活、更稀疏，能够针对不同任务，生成多种类型的推理（详见第 2 节）。

为了对比 ReAct 与 IM 的差异，并阐明内部推理相比简单的外部反馈反应的价值，我们开展了消融实验：采用类 IM 的密集型外部反馈思维模式构建模型。表 3 的结果显示，ReAct 的性能显著优于 IM 式提示模型（ReAct-IM），整体成功率分别为 71% 和 53%；且在 6 类任务中，ReAct 有 5 类均展现出持续优势。定性分析发现，由于缺乏对高层目标的拆解，ReAct-IM 常常无法正确判断子目标是否完成，也无法确定下一个子目标；此外，由于缺乏常识推理，许多 ReAct-IM 轨迹无法判断 ALFWorld 环境中物品的可能位置。而这两个缺陷，均能在 ReAct 范式中得到解决。ReAct-IM 的更多细节详见附录 B.2，提示词示例详见附录 C.4，轨迹示例详见附录 D.2.3。

5 相关工作

大语言模型的推理研究

利用大语言模型进行推理的最经典研究，是思维链（CoT）—— 该研究首次发现，大语言模型能够生成推理轨迹，解决算术、常识等推理问题。后续相关研究层出不穷，包括用于解决复杂任务的由浅入深提示法、零样本思维链，以及自洽性推理。近期，马丹与亚兹丹巴赫什对思维链的构成与结构进行了系统性研究，发现符号、模式和文本的结合，是思维链有效的关键。

还有研究突破了简单的提示学习，提出了更复杂的推理架构：例如选择 - 推理

（Selection-Inference）将推理过程拆解为 “选择” 和 “推理” 两个步骤；自举推理（STaR）通过在模型自身生成的正确推理过程上微调，实现推理能力的自举提升；可信推理（Faithful Reasoning）将多步推理拆解为三个步骤，每个步骤由专用的大语言模型完成。此外，草稿本（Scratchpad）等方法通过在中间计算步骤上对大语言模型进行微调，在多步计算问题上实现了性能提升。

与上述方法不同，ReAct 的核心并非孤立、固定的推理，而是将模型的行动及对应的观测信息，融入连贯的输入流中，让模型做出更准确的推理，并解决推理之外的任务（如交互式决策）。

大语言模型的决策研究

大语言模型的强大能力，使其能够完成语言生成之外的任务；将大语言模型作为策略模型应用于决策任务，尤其是交互式环境中的决策，正成为研究热点。WebGPT 利用大语言模型与浏览器交互、导航网页，并从 ELI5 数据集中推导复杂问题的答案。与 ReAct 不同，WebGPT 并未显式建模思考与推理过程，而是依靠高成本的人类反馈进行强化学习。

在对话建模领域，BlenderBot、Sparrow 等聊天机器人，以及 SimpleTOD 等面向任务的对话系统，也通过训练大语言模型，实现 API 调用的决策。这类方法同样未显式考虑推理过程，且需要高成本的数据集和人类反馈来学习策略。相比之下，ReAct 学习策略的成本更低 —— 其决策过程仅需用语言描述推理思路即可。

大语言模型也越来越多地被应用于交互式 and 具身环境的规划与决策，其中与 ReAct 最相关的研究是 SayCan 和《内心独白》，二者均将大语言模型用于机器人的行动规划与决策。SayCan 通过提示大语言模型，直接预测机器人可执行的行动，再由基于视觉环境的可用性模型重新排序，得出最终行动。《内心独白》则通过增加“内心独白”实现了性能提升，该独白由环境的反馈信息注入而成。据我们所知，《内心独白》是首个实现此类闭环系统的研究，ReAct 正是在其基础上进行的拓展。但我们认为，《内心独白》并非真正的“内部思维”，这一点在第 4 节详细阐述。

此外，已有研究证实，在交互式决策过程中，将语言作为语义丰富的输入是一种有效的策略。愈发明显的是，在大语言模型的助力下，语言作为一种基础的认知机制，将在交互与决策中发挥关键作用。不仅如此，大语言模型的发展还推动了通用智能体的研究。

6 结论

本研究提出了 ReAct——一种简洁且有效的方法，实现了大语言模型中推理与行动的协同。通过多跳问答、事实验证和交互式决策任务上开展大量实验，我们验证了 ReAct 能够在生成可解释决策轨迹的同时，实现性能的提升。尽管 ReAct 方法简洁，但面对具有大行动空间的复杂任务，模型需要更多的示例才能充分学习；而这一需求，很容易超出上下文学习的输入长度限制。我们在 HotPotQA 任务上探索了微调方法，取得了初步的积极结果；而利用更多高质量的人工标注数据进行学习，将是进一步提升 ReAct 性能的关键。将 ReAct 扩展至多任务训练，并与强化学习等互补范式相结合，有望构建性能更优的智能体，进一步释放大语言模型在更多实际应用中的潜能。

三、外文原文

REACT: SYNERGIZING REASONING AND ACTING IN LANGUAGE MODELS

Shunyu Yao^{*,1}, Jeffrey Zhao², Dian Yu², Nan Du², Izhak Shafran², Karthik Narasimhan¹, Yuan Cao²

1 INTRODUCTION

A unique feature of human intelligence is the ability to seamlessly combine task-oriented actions with verbal reasoning (or inner speech, Alderson-Day & Fernyhough, 2015), which has been theorized to play an important role in human cognition for enabling self-regulation or strategization (Vygotsky, 1987; Luria, 1965; Fernyhough, 2010) and maintaining a working memory (Baddeley, 1992). Consider the example of cooking up a dish in the kitchen. Between any two specific actions, we may reason in language in order to track progress (“now that everything is cut, I should heat up the pot of water”), to handle exceptions or adjust the plan according to the situation (“I don’t have salt, so let me use soy sauce and pepper instead”), and to realize when external information is needed (“how do I prepare dough? Let me search on the Internet”). We may also act (open a cookbook to read the recipe, open the fridge, check ingredients) to support the reasoning and to answer questions (“What dish can I make right now?”). This tight synergy between “acting” and “reasoning” allows humans to learn new tasks quickly and perform robust decision making or reasoning, even under previously unseen circumstances or facing information uncertainties.

Recent results have hinted at the possibility of combining verbal reasoning with interactive decision making in autonomous systems. On one hand, properly prompted large language models (LLMs)

have demonstrated emergent capabilities to carry out several steps of reasoning traces to derive

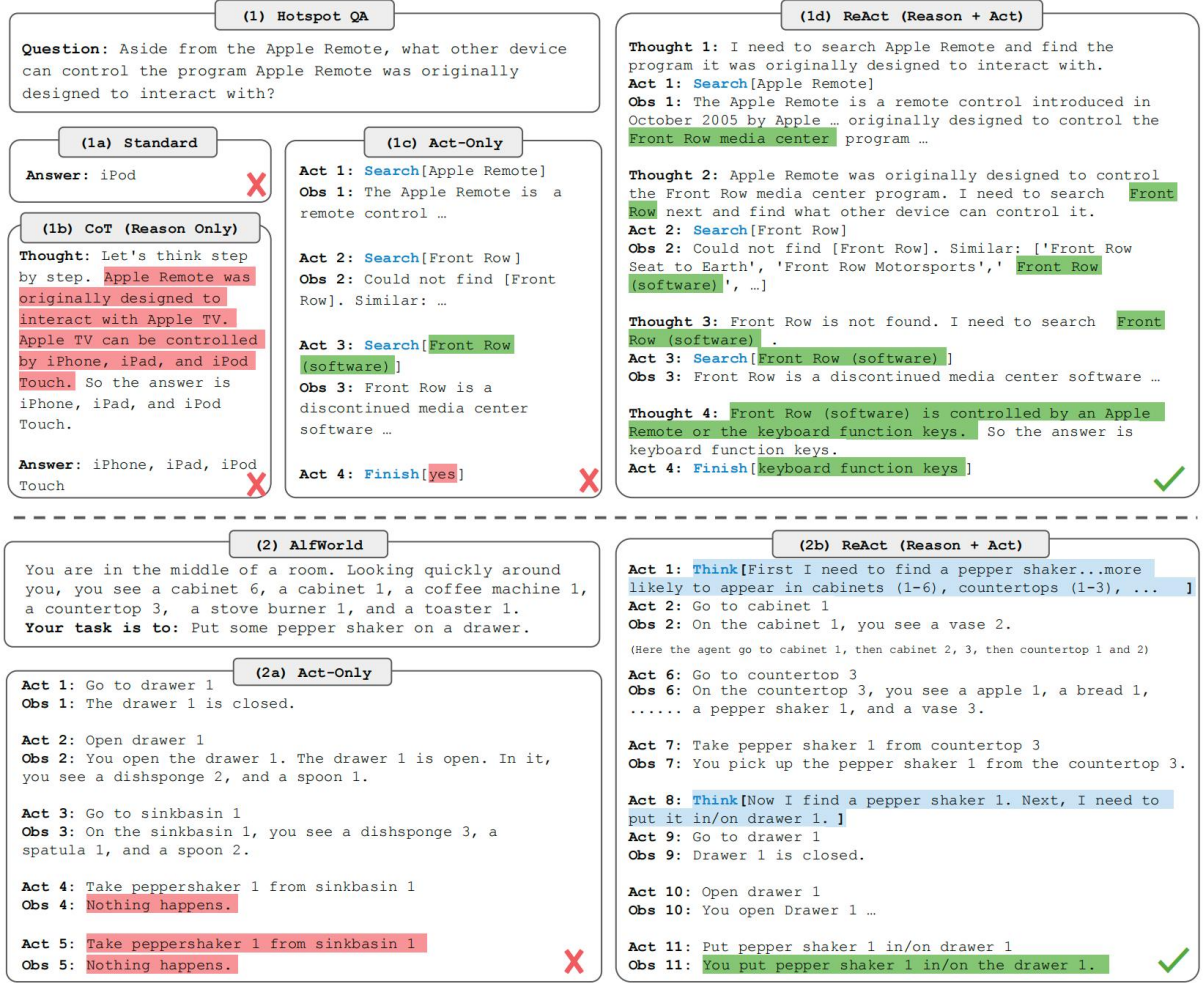


Figure 1: (1) Comparison of 4 prompting methods, (a) Standard, (b) Chain-of-thought (CoT, Reason Only), (c) Act-only, and (d) ReAct (Reason+Act), solving a HotpotQA (Yang et al., 2018) question. (2) Comparison of (a) Act-only and (b) ReAct prompting to solve an AlfWorld (Shridhar et al., 2020b) game. In both domains, we omit in-context examples in the prompt, and only show task solving trajectories generated by the model (Act, Thought) and the environment (Obs).

answers from questions in arithmetic, commonsense, and symbolic reasoning tasks (Wei et al., 2022). However, this “chain-of-thought” reasoning is a static black box, in that the model uses its own internal representations to generate thoughts and is not grounded in the external world, which limits its ability to reason reactively or update its knowledge. This can lead to issues like fact hallucination and error propagation over the reasoning process (Figure 1 (1b)). On the other hand, recent work has explored the use of pre-trained language models for planning and acting in

interactive environments (Ahn et al., 2022; Nakano et al., 2021; Yao et al., 2020; Huang et al., 2022a), with a focus on predicting actions via language priors. These approaches usually convert multi-modal observations into text, use a language model to generate domain-specific actions or plans, and then use a controller to choose or execute them. However, they do not employ language models to reason abstractly about high-level goals or maintain a working memory to support acting, barring Huang et al. (2022b) who perform a limited form of verbal reasoning to reiterate spatial facts about the current state. Beyond such simple embodied tasks to interact with a few blocks, there have not been studies on how reasoning and acting can be combined in a synergistic manner for general task solving, and if such a combination can bring systematic benefits compared to reasoning or acting alone.

In this work, we present ReAct, a general paradigm to combine reasoning and acting with language models for solving diverse language reasoning and decision making tasks (Figure 1). ReAct prompts LLMs to generate both verbal reasoning traces and actions pertaining to a task in an interleaved manner, which allows the model to perform dynamic reasoning to create, maintain, and adjust high-level plans for acting (reason to act), while also interact with the external environments (e.g. Wikipedia) to incorporate additional information into reasoning (act to reason).

We conduct empirical evaluations of ReAct and state-of-the-art baselines on four diverse benchmarks: question answering (HotPotQA, Yang et al., 2018), fact verification (Fever, Thorne et al., 2018), text-based game (ALFWorld, Shridhar et al., 2020b), and webpage navigation (WebShop, Yao et al., 2022). For HotPotQA and Fever, with access to a Wikipedia API that the model can interact with, ReAct outperforms vanilla action generation models while being competitive with chain-of thought reasoning (CoT) (Wei et al., 2022). The best approach overall is a combination of ReAct and CoT that allows for the use of both internal knowledge and externally obtained information during reasoning. On ALFWorld and WebShop, two or even one-shot ReAct prompting is able to outperform imitation or reinforcement learning methods trained with 103 ~ 105 task instances, with an absolute improvement of 34% and 10% in success rates respectively. We also demonstrate the importance of sparse, versatile reasoning

in decision making by showing consistent advantages over controlled baselines with actions only. Besides general applicability and performance boost, the combination of reasoning and acting also contributes to model interpretability, trustworthiness, and diagnosability across all domains, as humans can readily distinguish information from model’s internal knowledge versus external environments, as well as inspect reasoning traces to understand the decision basis of model actions.

To summarize, our key contributions are the following: (1) we introduce ReAct, a novel prompt based paradigm to synergize reasoning and acting in language models for general task solving; (2) we perform extensive experiments across diverse benchmarks to showcase the advantage of ReAct in a few-shot learning setup over prior approaches that perform either reasoning or action generation in isolation; (3) we present systematic ablations and analysis to understand the importance of acting in reasoning tasks, and reasoning in interactive tasks; (4) we analyze the limitations of ReAct under the prompting setup (i.e. limited support of reasoning and acting behaviors), and perform initial finetuning experiments showing the potential of ReAct to improve with additional training data. Scaling up ReAct to train and operate on more tasks and combining it with complementary paradigms like reinforcement learning could further unlock the potential of large language models.

2 REACT: SYNERGIZING REASONING + ACTING

Consider a general setup of an agent interacting with an environment for task solving. At time step t , an agent receives an observation $o_t \in \mathcal{O}$ from the environment and takes an action $a_t \in \mathcal{A}$ following some policy $\pi(a_t|c_t)$, where $c_t = (o_1, a_1, \dots, o_{t-1}, a_{t-1}, o_t)$ is the context to the agent. Learning a policy is challenging when the mapping $c_t \rightarrow a_t$ is highly implicit and requires extensive computation. For example, the agent shown in Figure 1(1c) is unable to generate the correct final action (Act 4) to finish the QA task as it requires complex reasoning over the trajectory context (Question, Act 1-3, Obs 1-3). Similarly, the agent shown in Figure 1(2a) fails to comprehend from the context that sinkbasin 1 does not contain peppershaker 1, thus keep producing hallucinating actions.

The idea of ReAct is simple: we augment the agent’s action space to $\hat{A} = A \cup L$, where L is the space of language. An action $\hat{a}^t \in L$ in the language space, which we will refer to as a thought or a reasoning trace, does not affect the external environment, thus leading to no observation feedback. Instead, a thought $\hat{a}^t$ aims to compose useful information by reasoning over the current context c_t , and update the context $c_{t+1} = (c_t, \hat{a}^t)$ to support future reasoning or acting. As shown in Figure 1, there could be various types of useful thoughts, e.g. decomposing task goals and create action plans (2b, Act 1; 1d, Thought 1), injecting commonsense knowledge relevant to task solving (2b, Act 1), extracting important parts from observations (1d, Thought2, 4), track progress and transit action plans (2b, Act 8), handle exceptions and adjust action plans (1d, Thought 3), and so on.

However, as the language space L is unlimited, learning in this augmented action space is difficult and requires strong language priors. In this paper, we mainly focus on the setup where a frozen large language model, PaLM-540B (Chowdhery et al., 2022)¹, is prompted with few-shot in-context examples to generate both domain-specific actions and free-form language thoughts for task solving (Figure 1 (1d), (2b)). Each in-context example is a human trajectory of actions, thoughts, and environment observations to solve a task instance (see Appendix C). For the tasks where reasoning is of primary importance (Figure 1(1)), we alternate the generation of thoughts and actions so that the task-solving trajectory consists of multiple thought-action-observation steps. In contrast, for decision making tasks that potentially involve a large number of actions (Figure 1(2)), thoughts only need to appear sparsely in the most relevant positions of a trajectory, so we let the language model decide the asynchronous occurrence of thoughts and actions for itself.

Since decision making and reasoning capabilities are integrated into a large language model, ReAct enjoys several unique features: A) Intuitive and easy to design: Designing ReAct prompts is straightforward as human annotators just type down their thoughts in language on top of their actions taken. No ad-hoc format choice, thought design, or example selection is used in this paper. We detail prompt design for each task in Sections 3 and 4. B) General and flexible: Due to the flexible thought space and thought-action occurrence format, ReAct works for diverse tasks with distinct action

spaces and reasoning needs, including but not limited to QA, fact verification, text game, and web navigation. C) Performant and robust: ReAct shows strong generalization to new task instances while learning solely from one to six in-context examples, consistently outperforming baselines with only reasoning or acting across different domains. We also show in Section 3 additional benefits when finetuning is enabled, and in Section 4 how ReAct performance is robust to prompt selections. D) Human aligned and controllable: ReAct promises an interpretable sequential decision making and reasoning process where humans can easily inspect reasoning and factual correctness. Moreover, humans can also control or correct the agent behavior on the go by thought editing, as shown in Figure 5 in Section 4.

3 KNOWLEDGE-INTENSIVE REASONING TASKS

We begin with knowledge-intensive reasoning tasks like multi-hop question answering and fact verification. As shown in Figure 1(1d), by interacting with a Wikipedia API, ReAct is able to retrieve information to support reasoning, while also use reasoning to target what to retrieve next, demonstrating a synergy of reasoning and acting.

3.1 SETUP

Domains

We consider two datasets challenging knowledge retrieval and reasoning: (1) HotPotQA (Yang et al., 2018), a multi-hop question answering benchmark that requires reasoning over two or more Wikipedia passages, and (2) FEVER (Thorne et al., 2018), a fact verification benchmark where each claim is annotated SUPPORTS, REFUTES, or NOT ENOUGH INFO, based on if there exists a Wikipedia passage to verify the claim. In this work, we operate in a question-only setup for both tasks, where models only receive the question/claim as input without access to support paragraphs, and have to rely on their internal knowledge or retrieve knowledge via interacting with an external environment to support reasoning.

Action Space

We design a simple Wikipedia web API with three types of actions to support interactive information retrieval: (1) `search[entity]`, which returns the first 5 sentences from the corresponding entity wiki page if it exists, or else suggests top-5 similar entities from the Wikipedia search engine, (2) `lookup[string]`, which would return the next sentence in the page containing string, simulating Ctrl+F functionality on the browser. (3) `finish[answer]`, which would finish the current task with answer. We note that this action space mostly can only retrieve a small part of a passage based on exact passage name, which is significantly weaker than state-of-the art lexical or neural retrievers. The purpose is to simulate how humans would interact with Wikipedia, and force models to retrieve via explicit reasoning in language.

3.2 METHODS

ReAct Prompting

For HotpotQA and Fever, we randomly select 6 and 3 cases² from the training set and manually compose ReAct-format trajectories to use as few-shot exemplars in the prompts. Similar to Figure 1(d), each trajectory consists of multiple thought-action-observation steps (i.e. dense thought), where free-form thoughts are used for various purposes. Specifically, we use a combination of thoughts that decompose questions (“I need to search x, find y, then find z”), extract information from Wikipedia observations (“x was started in 1844”, “The paragraph does not tell x”), perform commonsense (“x is not y, so z must instead be...”) or arithmetic reasoning (“ $1844 < 1989$ ”), guide search reformulation (“maybe I can search/look up x instead”), and synthesize the final answer (“...so the answer is x”). See Appendix C for more details.

Prompt Method ^a	HotpotQA (EM)	Fever (Acc)
Standard	28.7	57.1
CoT (Wei et al., 2022)	29.4	56.3
CoT-SC (Wang et al., 2022a)	33.4	60.4
Act	25.7	58.9
ReAct	27.4	60.9
CoT-SC $\rightarrow$ ReAct	34.2	64.6
ReAct $\rightarrow$ CoT-SC	35.1	62.0
Supervised SoTA^b	67.5	89.5

Table 1: PaLM-540B prompting results on HotpotQA and Fever.

^a HotpotQA EM is 27.1, 28.9, 33.8 for Standard, CoT, CoT-SC in (Wang et al., 2022b).

^b (Zhu et al., 2021) (Lewis et al., 2020)

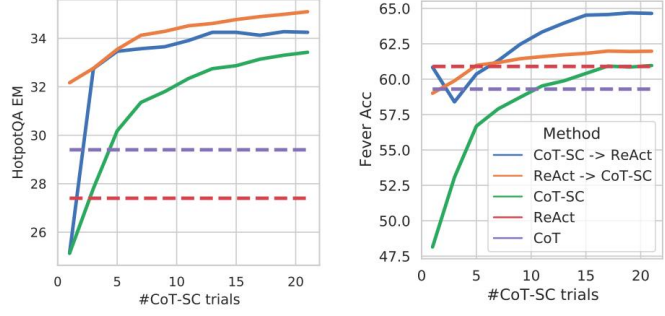


Figure 2: PaLM-540B prompting results with respect to number of CoT-SC samples used.

Baselines

We systematically ablate ReAct trajectories to build prompts for multiple baselines (with formats as Figure 1(1a-1c)): (a) Standard prompting (Standard), which removes all thoughts, actions, observations in ReAct trajectories. (b) Chain-of-thought prompting (CoT) (Wei et al., 2022), which removes actions and observations and serve as a reasoning-only baseline. We also build a self-consistency baseline (CoT-SC) (Wang et al., 2022a;b) by sampling 21 CoT trajectories with decoding temperature 0.7 during inference and adopting the majority answer, which is found to consistently boost performance over CoT. (c) Acting-only prompt (Act), which removes thoughts in ReAct trajectories, loosely resembling how WebGPT (Nakano et al., 2021) interacts with the Internet to answer questions, though it operates on a different task and action space, and uses imitation and reinforcement learning instead of prompting.

Combining Internal and External Knowledge

As will be detail in Section 3.3, we observe that the problem solving process demonstrated by ReAct is more factual and grounded, whereas CoT is more accurate in formulating reasoning structure but can easily suffer from hallucinated facts or thoughts. We therefore propose to incorporate ReAct and CoT-SC, and let the model decide when to switch to the other method based on the following heuristics: A) ReAct $\rightarrow$ CoT-SC: when ReAct fails to return an answer within given steps, back off to CoT-SC. We set 7 and 5 steps for HotpotQA and FEVER respectively as we find more steps will not improve ReAct performance³.

B) CoT-SC $\rightarrow$ ReAct: when the majority answer among n CoT-SC samples occurs less than $n/2$ times (i.e. internal knowledge might not support the task confidently), back off to ReAct.

Finetuning

Due to the challenge of manually annotating reasoning traces and actions at scale, we consider a bootstrapping approach similar to Zelikman et al. (2022), using 3,000 trajectories with correct answers generated by ReAct (also for other baselines) to finetune smaller language models (PaLM-8/62B) to decode trajectories (all thoughts, actions, observations) conditioned on input questions/claims. More details are in Appendix B.1.

3.3 RESULTS AND OBSERVATIONS

ReAct outperforms Act consistently

Table 1 shows HotpotQA and Fever results using PaLM- 540B as the base model with different prompting methods. We note that ReAct is better than Act on both tasks, demonstrating the value of reasoning to guide acting, especially for synthesizing the final answer, as shown in Figure 1 (1c-d). Fine-tuning results 3 also confirm the benefit of reasoning traces for more informed acting.

ReAct vs. CoT

On the other hand, ReAct outperforms CoT on Fever (60.9 vs. 56.3) and slightly lags behind CoT on HotpotQA (27.4 vs. 29.4). Fever claims for SUPPORTS/REFUTES might only differ by a slight amount (see Appendix D.1), so acting to retrieve accurate and up-to-date knowledge is vital. To better understand the behavioral difference between ReAct and CoT on HotpotQA, we randomly sampled 50 trajectories with correct and incorrect answers (judged by EM) from ReAct and CoT respectively (thus 200 examples in total), and manually labeled their success and failure modes in Table 2. Some key observations are as follows:

A) Hallucination is a serious problem for CoT, resulting in much higher false positive rate than ReAct (14% vs. 6%) in success mode, and make up its major failure mode (56%). In contrast, the problem solving trajectory of ReAct is more grounded, fact-driven, and trustworthy, thanks to the access of an external knowledge base.

B) While interleaving reasoning, action and observation steps improves ReAct’s groundedness and trustworthiness, such a structural constraint also reduces its flexibility in formulating reasoning steps, leading to more reasoning error rate than CoT. we note that there is one frequent error pattern specific to ReAct, in which the model repetitively generates the previous thoughts and actions, and we categorize it as part of “reasoning error” as the model fails to reason about what the proper next action to take and jump out of the loop⁴.

C) For ReAct, successfully retrieving informative knowledge via search is critical. Non informative search, which counts for 23% of the error cases, derails the model reasoning and gives it a hard time to recover and reformulate thoughts. This is perhaps an expected trade-off between factuality and flexibility, which motivates our proposed strategies of combining two methods. We provide examples for each success and failure modes in Appendix E.1. We also find some HotpotQA questions may contain outdated answer labels, see Figure 4 for example.

ReAct + CoT-SC perform best for prompting LLMs

Also shown in Table 1, the best prompting method on HotpotQA and Fever are ReAct → CoT-SC and CoT-SC → ReAct respectively. Furthermore, Figure 2 shows how different methods perform with respect to the number of CoT-SC samples used. While two ReAct + CoT-SC methods are advantageous at one task each, they both significantly and consistently outperform CoT-SC across different number of samples, reaching CoT-SC performance with 21 samples using merely 3-5 samples. These results indicate the value of properly combining model internal knowledge and external knowledge for reasoning tasks. **ReAct performs best for**

fine-tuning

Figure 3 shows the scaling effect of prompting/finetuning four methods (Standard, CoT, Act, ReAct) on HotpotQA. With PaLM-8/62B, prompting ReAct performs worst among four methods due to the difficulty to learn both reasoning and acting from in-context examples. However, when finetuned with just 3,000 examples, ReAct becomes the best method among the four, with PaLM-8B finetuned ReAct outperforming all PaLM-62B prompting methods, and PaLM-62B finetuned ReAct outperforming all 540B prompting methods. In contrast, finetuning Standard or CoT is significantly worse than finetuning ReAct or Act for both PaLM- 8/62B, as

the former essentially teaches models to memorize (potentially hallucinated) knowledge facts, and the latter teaches models how to (reason and) act to access information from Wikipedia, a more generalizable skill for knowledge reasoning. As all prompting methods are still significantly far from domain-specific state-of-the-art approaches (Table 1), we believe finetuning with more human-written data might be a better way to unleash the power of ReAct.

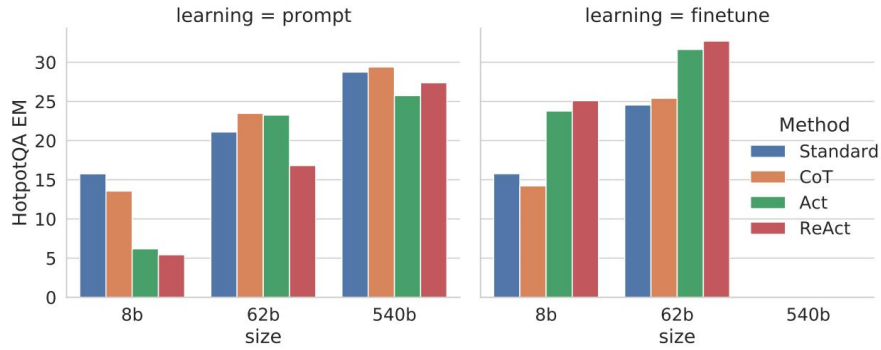


Figure 3: Scaling results for prompting and finetuning on HotPotQA with ReAct (ours) and baselines.

4 DECISION MAKING TASKS

We also test ReAct on two language-based interactive decision-making tasks, ALFWorld and WebShop, both of which feature complex environments that require agents to act over long horizons with sparse rewards, warranting the need for reasoning to act and explore effectively.

ALFWorld

ALFWorld (Shridhar et al., 2020b) (Figure 1(2)) is a synthetic text-based game designed to align with the embodied ALFRED benchmark (Shridhar et al., 2020a). It includes 6 types of tasks in which an agent needs to achieve a high-level goal (e.g. examine paper under desk lamp) by navigating and interacting with a simulated household via text actions (e.g. go to coffeetable 1, take paper 2, use desk lamp 1). A task instance can have more than 50 locations and take an expert policy more than 50 steps to solve, thus challenging an agent to plan and track subgoals, as well as explore systematically (e.g. check all desks one by one for desk lamp). In particular, one challenge built into ALFWorld is the need to determine likely locations for common household items (e.g. desk lamps will likely be on desks, shelves, or dressers), making this

environment a good fit for LLMs to exploit their pretrained commonsense knowledge. To prompt ReAct, we randomly annotate three trajectories from the training set for each task type, where each trajectory includes sparse thoughts that (1) decompose the goal, (2) track subgoal completion, (3) determine the next subgoal, and (4) reason via commonsense where to find an object and what to do with it. We show prompts used for ALFWorld in Appendix C.4.

Following Shridhar et al. (2020b), we evaluate on 134 unseen evaluation games in a task-specific setup. For robustness, we construct 6 prompts for each task type through each permutation of 2 annotated trajectories from the 3 we annotate. Act prompts are constructed using the same trajectories, but without thoughts — since task instances are randomly chosen from the training set, it favors neither ReAct nor Act and provides a fair and controlled comparison to test the importance of sparse thoughts. For baselines, we use BUTLER (Shridhar et al., 2020b), an imitation learning agent trained on 105 expert trajectories for each task type⁵.

WebShop

Can ReAct also interact with noisy real-world language environments for practical applications?

We investigate WebShop (Yao et al., 2022), a recently proposed online shopping website environment with 1.18M real-world products and 12k human instructions. Unlike ALFWorld, Webshop contains a high variety of structured and unstructured texts (e.g. product titles, descriptions, and options crawled from Amazon), and requires an agent to purchase a product based on a user instruction (e.g. “I am looking for a nightstand with drawers. It should have a nickel finish, and priced lower than \$140”) through web interactions (e.g. search “nightstand drawers”, choose buttons such as “color: modern-nickel-white” or “back to search”). This task is evaluated by average score (percentage of desired attributes covered by the chosen product averaged across all episodes) and success rate (percentage of episodes where the chosen product satisfies all requirements) on 500 test instructions. We formulate Act prompts with actions to search, choose product, choose options, and buy, with ReAct prompts additionally reasoning to determine what to explore, when to buy, and what products options are relevant to the instruction. See Table 6 for an example prompt, and Table 10 for model predictions in the Appendix. We compare to an imitation learning (IL) method

Results

ReAct outperforms Act on both ALFWorld (Table 3) and Webshop (Table 4). On ALFWorld, the best ReAct trial achieves an average success rate of 71%, significantly outperforming the best Act (45%) and BUTLER (37%) trials. In fact, even the worse ReAct trial (48%) beats the best trial of both methods. Moreover, the advantage of ReAct over Act is consistent across six controlled trials, with relative performance gain ranging from 33% to 90% and averaging 62%. Qualitatively, we saw that, without any thoughts at all, Act fails to correctly decompose goals into smaller subgoals, or loses track of the current state of the environment. Example trajectories comparing ReAct and Act can be found in Appendix D.2.1 and Appendix D.2.2. On Webshop, one-shot Act prompting already performs on par with IL and IL+RL methods. With additional sparse reasoning, ReAct achieves significantly better performance, with an absolute 10% improvement over the previous best success rate. By checking examples, we find that ReAct is more likely to identify instruction-relevant products and options by reasoning to bridge the gap between noisy observations and actions (e.g. “For ‘space-saving ottoman bench for living room’, the item has options ‘39x18x18inch’ and ‘blue’ and seems good to buy.”). However, existing methods are still far from the performance of expert humans (Table 4), who perform significantly more product explorations and query re-formulations that are still challenging for prompting-based methods. On the value of internal reasoning vs. external feedback To our knowledge, ReAct is the first demonstration of combined reasoning and action using an LLM applied to an interactive environment within a closed-loop system. Perhaps the closest prior work is Inner Monologue (IM), from Huang et al. (2022b), in which actions from an embodied agent are motivated by an eponymous “inner monologue”. However, IM’s “inner monologue” is limited to observations of the environment state and what needs to be completed by the agent for the goal to be satisfied. In contrast, the reasoning traces in ReAct for decision making is flexible and sparse, allowing diverse reasoning types (see Section 2) to be induced for different tasks. To demonstrate the differences between ReAct and IM, and to highlight the importance of internal reasoning vs. simple reactions to external feedback, we ran an ablation experiment

using a thought pattern composed of IM-like dense external feedback. As can be seen in Table 3, ReAct substantially outperforms IM-style prompting (ReAct-IM) (71 vs. 53 overall success rate), with consistent advantages on five out of six tasks. Qualitatively, we observed that ReAct-IM often made mistakes in identifying when subgoals were finished, or what the next subgoal should be, due to a lack of high level goal decomposition. Additionally, many ReAct-IM trajectories struggled to determine where an item would likely be within the ALFWorld environment, due to a lack of commonsense reasoning.

Both shortcomings can be addressed in the ReAct paradigm. More details about ReAct-IM is in Appendix B.2. An example prompt for ReAct-IM can be found in Appendix C.4, and an example trajectory in Appendix D.2.3.

5 RELATED WORK

Language model for reasoning

Perhaps the most well-known work of using LLMs for reasoning is Chain-of-Thought (CoT) (Wei et al., 2022), which reveals the ability of LLMs to formulate their including least-to-most prompting for solving complicated tasks (Zhou et al., 2022), zero-shot CoT (Kojima et al., 2022), and reasoning with self-consistency (Wang et al., 2022a).

Recently, (Madaan & Yazdanbakhsh, 2022) systematically studied the formulation and structure of CoT, and observed that the presence of symbols, patterns and texts is crucial to the effectiveness of CoT. Other work has also been extended to more sophisticated reasoning architecture beyond simple prompting.

For example Selection-Inference (Creswell et al., 2022) divides the reasoning process into two steps of “selection” and “inference”. STaR (Zelikman et al., 2022) bootstraps the reasoning process by finetuning the model on correct rationales generated by the model itself. Faithful reasoning (Creswell & Shanahan, 2022) decomposes multi-step reasoning into three steps, each performed by a dedicated LM respectively. Similar approaches like Scratchpad (Nye et al., 2021), which finetunes a LM on intermediate computation steps, also demonstrate improvement on multi-step computation problems. In contrast to these methods, ReAct performs more than

just isolated, fixed reasoning, and integrates model actions and their corresponding observations into a coherent stream of inputs for the model to reason more accurately and tackle tasks beyond reasoning (e.g. interactive decision making).

Language model for decision making

The strong capability of LLMs has enabled them to perform tasks beyond language generation, and it is becoming more popular to take advantage of LLMs as a policy model for decision making, especially in interactive environments. WebGPT (Nakano et al., 2021) uses an LM to interact with web browsers, navigate through web pages, and infer answers to complicated questions from ELI5 (Fan et al., 2019). In comparison to ReAct, WebGPT does not explicitly model the thinking and reasoning procedure, instead rely on expensive human feedback for reinforcement learning. In conversation modeling, chatbots like BlenderBot (Shuster et al., 2022b) and Sparrow (Glaese et al., 2022) and task-oriented dialogue systems like SimpleTOD (Hosseini-Asl et al., 2020) also train LMs to make decision about API calls. Unlike ReAct, they do not explicitly consider the reasoning procedure either, and also relies on expensive datasets and human feedback collections for policy learning. In contrast, ReAct learns a policy in a much cheaper way, since the decision making process only requires language description of the reasoning procedure.⁶ LLMs have also been increasingly employed in interactive and embodied environments for planning and decision making. Perhaps most relevant to ReAct in this respect are SayCan (Ahn et al., 2022) and Inner Monologue (Huang et al., 2022b), which use LLMs for robotic action planning and decision making. In SayCan, LLMs were prompted to directly predict possible actions a robot can take, which is then reranked by an affordance model grounded on the visual environments for final prediction. Inner Monologue made further improvements by adding the eponymous “inner monologue”, which is implemented as injected feedback from the environment. To our knowledge, Inner Monologue is the first work that demonstrates such a closed-loop system, which ReAct builds on. However, we argue that Inner Monologue does not truly comprise of inner thoughts — this is elaborated in Section 4. We also note that leveraging language as semantically-rich inputs in the process of interactive decision making has been shown to be successful under other settings

(Abramson et al., 2020; Karamcheti et al., 2021; Huang et al., 2022a; Li et al., 2022). It is becoming more evident that with the help of LLMs, language as a fundamental cognitive mechanism will play a critical role in interaction and decision making. What is more, progress in LLMs has also inspired the development of versatile and generalist agents like Reed et al. (2022).

6 CONCLUSION

We have proposed ReAct – a simple yet effective method for synergizing reasoning and acting in large language models. Through a diverse set of experiments on multi-hop question-answering, fact checking, and interactive decision-making tasks, we show that ReAct leads to superior performance with interpretable decision traces. Despite the simplicity of our method, complex tasks with large action spaces require more demonstrations to learn well, which unfortunately can easily go beyond the input length limit of in-context learning. We explore the fine-tuning approach on HotpotQA with initial promising results, but learning from more high-quality human annotations will be the desiderata to further improve the performance. Scaling up ReAct with multi-task training and combining it with complementary paradigms like reinforcement learning could result in stronger agents that further unlock the potential of LLMs for more applications.

毕业设计开题报告、外文翻译的考核

导师对开题报告、外文翻译的评语及成绩评定：

成绩比例	开题报告 (15%)	外文翻译 (5%)
分 值		

导师签名_____

年 月 日

学院盲审专家对开题报告、外文翻译的评语及成绩评定：

成绩比例	开题报告 (15%)	外文翻译 (5%)
分 值		

开题报告审核负责人（签名/签章）_____

年 月 日

毕业设计中期报告考核

导师对中期报告的评语及成绩评定：

成绩比例	中期报告 (10%)
分 值	

导师签名_____

年 月 日